技术部第五次技术分享会

枚举、泛型与常用标准库

姚雨森　技术部分管主席

用整数表示状态，需要记住编号的含义

一个订单可能处于待付款、已付款、已发货或已取消状态。

```
int status = 0;

if (status == 1) {
    System.out.println("Ready to ship");
}

status = 7;
```

- 阅读代码时，需要另外查找：0 和 1 分别代表什么？
- int 允许许多整数，但业务只需要四种状态。
- 即使 7 没有业务含义，赋值语句仍能通过类型检查。

换成字符串更直观，但仍然可能写错

```
String status = "PENDING";
status = "PAID";

status = "PAIID";
status = "paid";
```

- 名称比编号容易理解，但任何字符串都可以赋给 status。
- 拼写错误、大小写不一致，都可能让后面的判断失效。
- 我们希望同时表达每个选项的含义和允许使用的选项。

枚举：为有限的一组选择定义类型

订单状态只需要以下四个选项：

含义	枚举常量名称
待付款	PENDING
已付款	PAID
已发货	SHIPPED
已取消	CANCELLED

枚举（enum）将这些选项列在一起，定义一个专门的类型。

星期、方向、任务状态等有限选项，也可以用枚举表达。

使用 enum 定义订单状态类型

将下面的定义放在 OrderStatus.java 中：

```
public enum OrderStatus {
    PENDING,
    PAID,
    SHIPPED,
    CANCELLED
}
```

- enum 用于声明枚举；OrderStatus 是新类型的名称。
- 花括号中列出枚举常量，使用逗号分隔。
- 类型名称通常使用大驼峰；常量通常使用大写字母和下划线。
- 使用常量时写成类型名. 常量名，例如 OrderStatus.PAID。

像其他类型一样声明变量和赋值

以下语句写在 Main.java 的 main 方法中，两个源文件放在同一目录。

```
OrderStatus status = OrderStatus.PENDING;
System.out.println(status); // PENDING

status = OrderStatus.PAID;
System.out.println(status); // PAID
```

- 左边的 OrderStatus 指定变量类型。
- 右边的 OrderStatus.PENDING 是该类型的一个常量。
- 变量可以重新赋值；这里是改为保存另一个枚举常量。

类型检查帮助我们排除无效的赋值

下面的错误写法用于说明类型限制，不能通过编译：

```
OrderStatus status = OrderStatus.PENDING;

status = 1;                    // Wrong type
status = "PAID";               // Wrong type
status = OrderStatus.PAIID;    // No such constant
```

- 枚举常量有自己的类型；字符串 "PAID" 并不是它。
- 拼错常量名称时，编译器就能指出问题。
- 另一种枚举类型的常量，即使同名，也不能赋给这个变量。

补充：枚举变量也可以为 null，表示没有引用枚举常量；本例始终给它赋明确的状态。

使用 == 判断是否为某个枚举常量

```
OrderStatus status = OrderStatus.PAID;

if (status == OrderStatus.PAID) {
    System.out.println("Ready to ship");
}

if (status != OrderStatus.CANCELLED) {
    System.out.println("Order is not cancelled");
}
```

- == 判断是否为同一个枚举常量；!= 判断是否不同。
- 直接比较枚举值，不需要先转换成编号或字符串。
- 字符串内容比较仍使用 equals，不要把这里的规则照搬过去。

枚举作为参数：明确方法接收哪一种状态

假设本例规定：只有待付款的订单可以取消。

在 Main 类中定义方法：

```
static boolean canCancel(OrderStatus status) {
    return status == OrderStatus.PENDING;
}
```

在 main 方法中调用：

```
boolean first = canCancel(OrderStatus.PENDING); // true
boolean second = canCancel(OrderStatus.SHIPPED); // false
```

参数的类型说明需要什么数据；方法体决定这组数据对应的业务规则。

商品价格需要满足非负的要求

回顾：通过 setter 修改商品的价格。本例价格用整数表示，单位为分。

```
public class Product {
    private int price;

    public void setPrice(int price) {
        this.price = price;
    }
}
```

- 字段设为 `private`，类外通过 `setPrice` 修改它。
- 目前调用 `setPrice(-10)`，仍然会把价格改为负数。
- 我们希望拒绝非法价格，并让调用方知道操作失败。

使用 throw 抛出异常，拒绝非法参数

将 Product 中的 setter 改为：

```
public void setPrice(int price) {
    if (price < 0) {
        throw new IllegalArgumentException(
            "Price must be non-negative");
    }
    this.price = price;
}
```

- IllegalArgumentException 表示参数不符合方法的要求。
- new 创建异常对象；字符串说明失败的原因。
- throw 抛出异常，本次调用不能正常完成。
- 检查在赋值之前：负数不会写入字段，0 则是合法价格。

异常类型帮助我们识别失败原因

异常类型	典型原因
`IllegalArgumentException`	参数不符合要求
`NumberFormatException`	文本不能转换为要求的数值
`NullPointerException`	对 `null` 进行需要对象的操作
`IndexOutOfBoundsException`	索引超出允许范围

- 参数校验失败时，可以主动抛出合适的异常。
- 调用库方法或执行其他操作时，也可能出现异常。
- 类型说明问题的类别，异常消息提供本次失败的具体信息。

异常没有被处理，调用方也不能照常继续

在 main 方法中调用修改后的 setter：

```
Product product = new Product();
product.setPrice(1000);
product.setPrice(-10);
System.out.println("Price updated");
```

- 第一次设置成功；第二次在 setter 中抛出异常。
- 异常传播到 main，最后的输出语句不会执行。
- 始终没有匹配的处理时，当前线程终止；本例程序随之结束。

控制台会给出异常类型、消息和调用堆栈；可以从中找到自己代码的文件名与行号。

在调用方使用 try-catch 处理设置失败

以下代码写在 main 方法中：

```
Product product = new Product();
product.setPrice(1000);
try {
    product.setPrice(-10);
    System.out.println("Price updated");
} catch (IllegalArgumentException e) {
    System.out.println(e.getMessage());
}
System.out.println("Continue");
```

- try 中放置可能失败的操作。
- catch 捕获匹配的异常，变量 e 接收异常对象。
- e.getMessage() 取得异常消息，此处用它给出失败提示。

对照两条路径：正常执行与异常处理

将上一页的 `-10` 分别改为 `2000` 和 `-10`：

传入 2000：正常完成

1. 设置价格成功。
2. 输出 `Price updated`。
3. 跳过 `catch`。
4. 输出 `Continue`。

传入 -10：进入异常处理

1. setter 抛出异常。
2. 跳过 `try` 中剩余语句。
3. 进入匹配的 `catch`，输出原因。
4. 输出 `Continue`。

本例处理后，继续执行整个 `try-catch` 后面的语句，不会回到出错位置接着执行。

setter 检查规则，调用方处理失败

- **setter：**检查价格，拒绝负数，并抛出异常报告失败。
- **调用方：**根据场景提示用户、重新输入，或取消这次操作。
- **自己的代码写错时：**修正非法参数的来源，避免反复触发同一问题。

捕获异常不会自动修复数据，也不会自动重试

本例价格仍为原来的 `1000`，是因为 setter 先检查、后赋值。异常机制本身不会撤销此前已经完成的修改。

不要写空的 `catch` 来隐藏失败；在能作出合理处理的位置捕获异常。

补充：有些异常要求捕获或显式声明

- Java 对部分异常有编译要求：调用方必须捕获，或在方法上用 `throws` 声明可能向外传播。
- 本例的 `IllegalArgumentException` 没有这项强制要求，也可以使用 `try-catch` 处理。

区分两个关键字

`throw`：实际抛出异常。

`throws`：写在方法声明中，说明可能向外传播的异常，不代表已经处理了异常。

课后了解：受检异常与非受检异常的分类，以及 `throws` 的使用。